

SeedHammer II — the composer journey

A wallet policy *built on the device*, from the operator's own keys, and proved before it reaches steel.

Every other journey in this repo hands the device a wallet somebody built elsewhere. This one builds it *on* the SeedHammer II: the operator picks a script type, lists spend paths (keys, a wait, a hashlock), seats keys from a systemwide payload, and the device lowers the list to a BIP-388 template by the same fixed rules `md` `compose` uses on the host. The point of the walk is that **three implementations meet on the same bytes**: the host (`md`, `me`, `ms`, `mk`), the firmware on the emulator, and — for the key-less arm — the string on the one plate the operator cuts.

The wallet

Segwit (wsh). Path 1: 2-of-2. Path 2: one key, after a wait of 12960 blocks, with a sha256 hashlock. Lowered, that is:

[illegible]

Nothing secret is committed. Master A is BIP-39's "abandon ... about" vector and master B its "legal winner ... yellow" vector. **Path 1's two keys are two ACCOUNTS of master A, deliberately:** it is the "one master, two accounts" case the door labels for, and it makes the mapping review fire its *same seed, same path* warning — a §8 body no other fixture reaches, photographed on page 4. Do not read that warning as a defect, and never put funds behind these keys.

The payload

Five records, packed with me sysw pack: two key: records (A's accounts 0 and 1), a hash: record, a now: record (the pack time, a lower bound the device echoes beside a time lock), and master B's words as a seed.

[illegible]

```
payload digestdigest: db e9 e774 e9a4 9231 0b62 626c 2b41 cf4b
```

[illegible]

```
$ diff <(/scratch/code/shibboleth/.toolchain/go/bin/go run ./cmd/buildpayloadcomposer) /scratch/code/shibboleth/wt-engrave-s
4-emu/design/journeys/out/composer/records.txt
[exit 0]
GATE PASS    records.txt == cmd/buildpayloadcomposer stdout
              = identical
```

```
GATE PASS Template-ID
          = 531ab9e1777f018ae53694387dd0d128
GATE PASS Policy-ID
          = 4dd749a8372af515a61d7104faf944ef
GATE PASS md1-encoding-id
          = fb28698ee8dbdc18c6ee36598f2124fe
```

The device's mk1 stubs are the FIRST FOUR BYTES of those two ids:

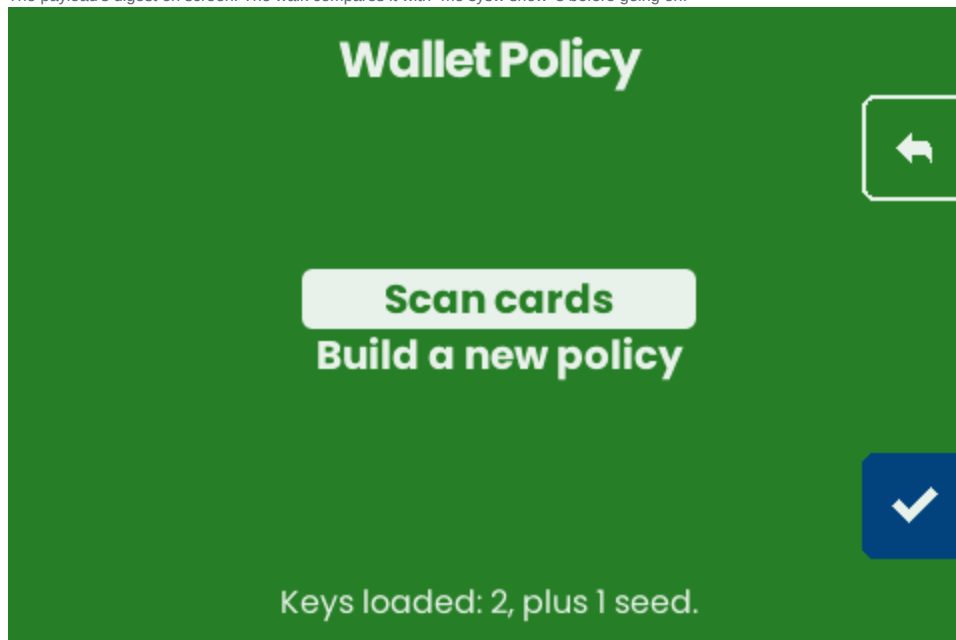
[illegible]

```
receive 0..1bc1q8cf5g5fxfld9t22xguk7e0mg9mkjl2ujcxuux9napkw8cy89n3mqk0tp4l
bc1qkd729k2r3kvrewzgdtpj0quhrrrv9u4jgndt2zsmyp6ypnr7rslzwsfhmu9a
change 0..1bc1q9ms8tdk54dzaelef0rrrg82fpm3s9nfgyr30aed96rnyuj02hhgrqy3dyru
bc1q3cs923r9rdcv5s8zmmkd5strrh7svzpzpg2yr14hcue3f3fv4lyfsdp3tz9
```

3 · The device: loading, the door, the shape



The payload's digest on screen. The walk compares it with `me sysw show`'s before going on.



Wallet Policy's door states the key state: two keys and a seed are loaded. There is no 'From payload' row -- the payload holds no finished wallet.



'Start from?' opens on 'Build my own paths' (the S4 walk's W-1), then the six presets.



Path 2's wait, echoed in the operator's units. A relative lock carries no pack-time bound line.

Path 2 hash

The hash must be SHA-256 of a 32-byte value. A passphrase must be hashed to 32 bytes first, then hashed again. A hash of the passphrase itself can never be spent.



Choosing the payload's hashlock first shows the hash rule: a preimage must be a 32-byte value hashed once more; a hashed passphrase can never be spent.

Template

Template-ID: 53lab9e1777f018ae53694387dd0d128
mk1 stub (template): 53lab9e1



mk encode --xpub <xpub> --origin-fingerprint <fp>
--origin-path <path> --policy-id-stub 53lab9e1



The template screen: the key-independent Template-ID, the mk1 stub every key card must carry, and the mk encode line to mint one. As captured here the Template-ID's last hex digit sits under the Back button -- the S4 walk's W-3, fixed on the fork after this run.

4 · Seating, and the mapping review

Seat keys

Slot @0, Path 1 key 1 of 2: choose a key



73c5da0a m/48h/0h/0h/2h

73c5da0a m/48h/0h/1h/2h

Type a seed

Leave unseated



Slot @0 offers the payload's two key records by fingerprint and origin, plus 'Type a seed' and 'Leave unseated'.

Seat keys

Slot @2, Path 2 key 1 of 1: choose a key



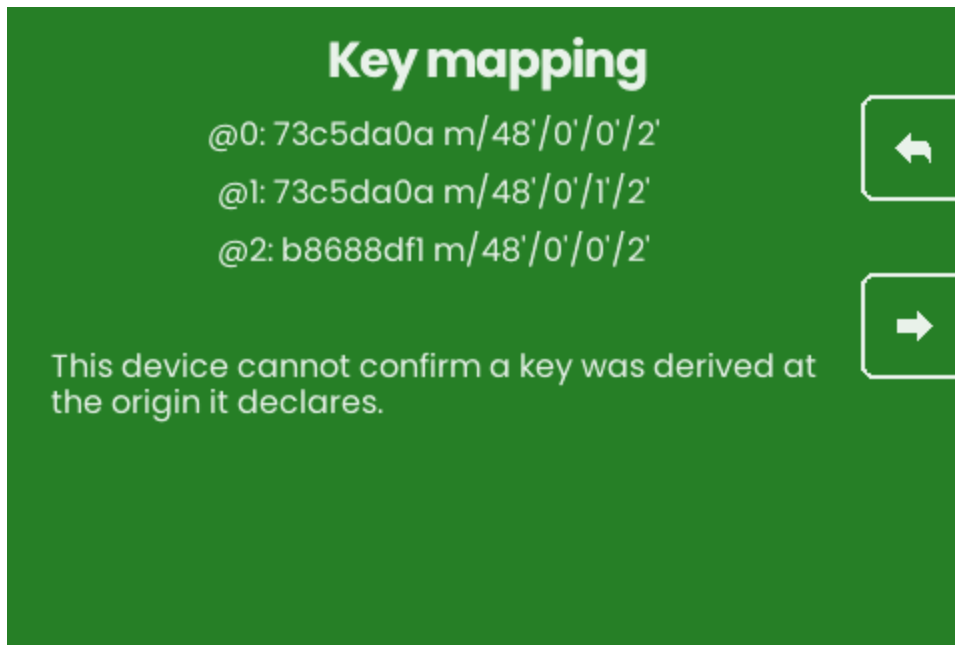
seed 1 (any slots)

Type a seed

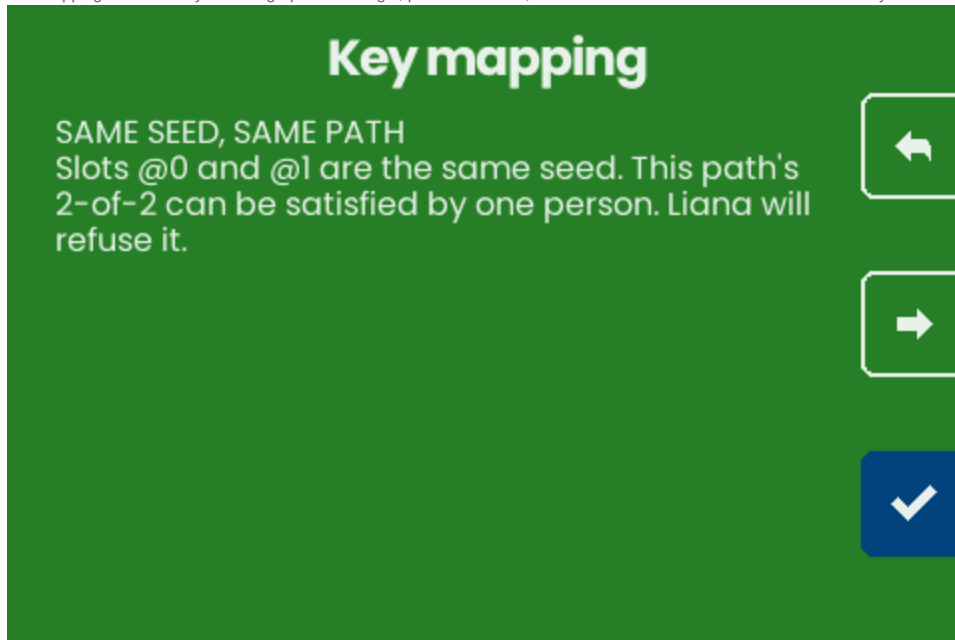
Leave unseated



Slot @2 is filled from the payload's seed (master B), offered as 'seed 1 (any slots)' after 'Type a seed' -> 'FROM PAYLOAD'.



The mapping review: every slot's fingerprint and origin, printed verbatim, with the line that the device cannot confirm a key was derived there.



Page 2: 'same seed, same path' -- Path 1's 2-of-2 is two accounts of one master and can be satisfied by one person. Deliberate here; see page 1.

The device seats the seed's slot at **master B's own account 0'** (m/48'/0'/0'/2'), not at the lowest account free in the template. The host oracle was minted with that origin, and the Policy-ID on the next page is the proof that the two rules agree.

5 · The proof, on the consent path

Template

Template-ID: 53lab9e1777f018ae53694387dd0d128
mk1 stub (template): 53lab9e1
Policy-ID: 4dd749a8372af515a61d7104faf944ef
mk1 stub (policy): 4dd749a8
Stamp BOTH stubs on each key card:
--policy-id-stub 53lab9e1 --policy-id-stub 4dd749a8



template screen after seating, page 1: both ids and both stubs

Template

mk encode --xpub <xpub> --origin-fingerprint <fp>
--origin-path <path> --policy-id-stub 53lab9e1

A wallet built here is its own wallet. The same rules written by another tool give a different id and different addresses.

Slot @0: 73c5da0a m/48h/0h/0h/2h



template screen after seating, page 2: both ids and both stubs

Review

Path 1: 2-of-2

Path 2: 1 key

12960 blocks (about 90.0 days)

hash abababab..abababab

Policy-ID: 4dd749a8372af515a61d7104faf944ef



consent, page 1

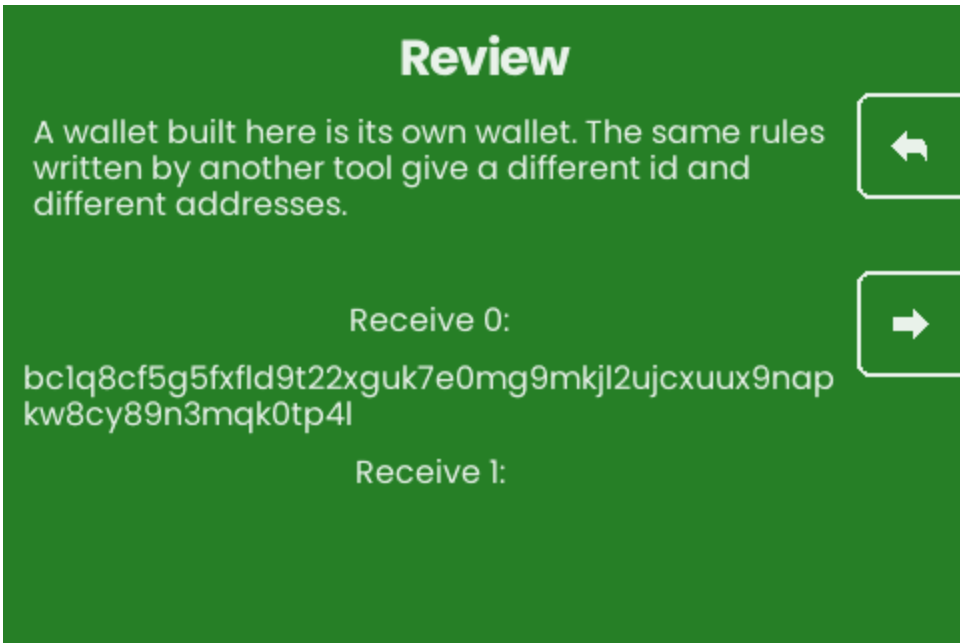
Review

mk1 stub (policy): 4dd749a8

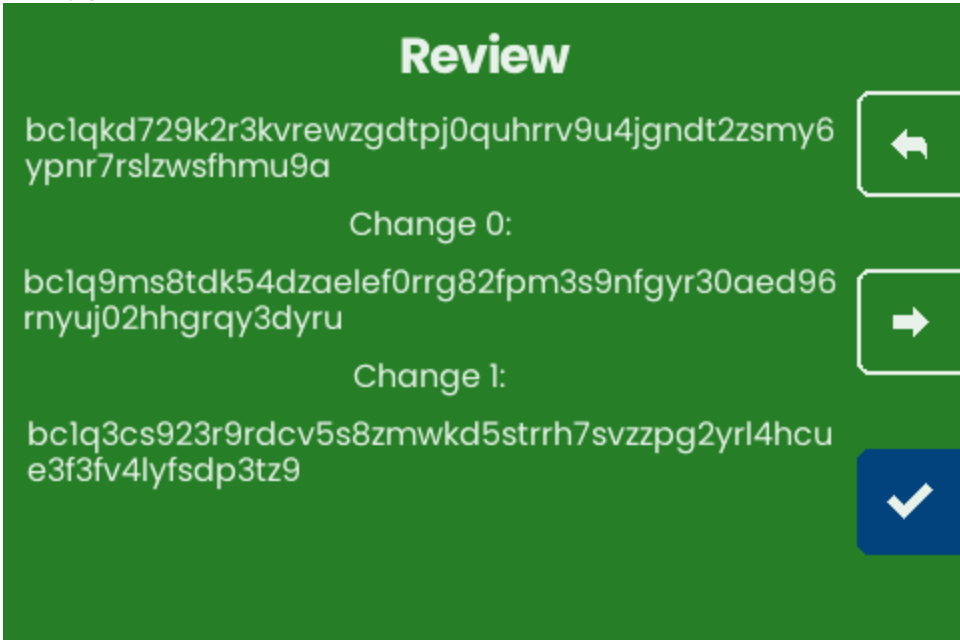
The hash must be SHA-256 of a 32-byte value. A passphrase must be hashed to 32 bytes first, then hashed again. A hash of the passphrase itself can never be spent.



consent, page 2



consent, page 3



consent, page 4

Template-ID531ab9e1777f018ae53694387dd0d128

mk1 stub (template)531ab9e1

Policy-ID4dd749a8372af515a61d7104faf944ef

mk1 stub (policy)4dd749a8

addresses matchedbc1q8cf5g5fxfld9t22xguk7e0mg9mkjl2ujcxuux9napkw8cy89n3mqk0tp4l

bc1qkd729k2r3kvrewzgdtpj0quhrrv9u4jgndt2zsmly6ypnr7rslzwsfhmu9a

bc1q9ms8tdk54dzaelef0rrg82fpm3s9nfgyr30aed96rnyuj02hhgrqy3dyru

bc1q3cs923r9rdcv5s8zmkwd5strrh7svzzpg2yrl4hcu e3f3fv4lyfsdp3tz9

Both ids are **named** on the screen (the key-stable Template-ID and the key-dependent Policy-ID look identical as 16 bytes of hex), and the four addresses are lines the operator must page through to engrave.

Plates To Cut

This engraves 2 plates.

mdl policy: 2 plates (the wallet policy, with its keys,

Each plate takes minutes to cut. Have that many blanks ready before you start: a set is only a backup when all of it exists.

mdl and mkl plates carry error correction. A plain descriptor plate carries only its checksum, which finds a mistake but cannot fix one.

Plates To Cut

This engraves 4 plates.

mdl template: 1 plate (key-less wallet policy)

mk1 key @0: 1 plate (m/48'/0'/0'/2')

mk1 key @1: 1 plate (m/48'/0'/1'/2')

mk1 key @2: 1 plate (m/48'/0'/0'/2')

Each plate takes minutes to cut. Have that many blanks ready before you start: a set is only a backup when all exists.

```
md1flv5xr9q9jtvyvy5jmprrjjtvyvy4qqxpqcyy2v6tjv6a4w46h2at4w46h2at4w46h2at4g9f8q03ncnsuxv
md1flv5xr9q46h2at4w46h2at4w46h2at4w46hqqq2qzlxrncdhdq5h83w6p2hp5gmug4u83waqgk86n6tsjqqddg
md1flv5xr9qsc19pvz58pmltjst9tjrg0g2z0agd4urfpzanhaq3lcdlz64mrqgdrrha0m7umapumg67j7fj6m3fvh7y
md1flv5xr9qej075dzhfzvynh66n94j5lclmx9ayav9mj0jjejxcy50llpx82qfmryv7l68w6hqhpyxt9s9j68n
md1flv5xrpzragnj3g5vrl85zeape8w0dvczfy55tqsd5576trsa3p40nfpd7hsyjjf7vlx6saxqugf5v7er72p
md1flv5xrp0k2j6ckr4wf0m36nn9pm0wkz5duhr4fq2pwsjch4zfmclsyyxap2w2ua7583pn5tsnkrk4cfxj37uw
md1flv5xrp7jqeeyp4dfykwfkgg6fxyxetdcmtythf4hsqzd3v879jprztejs7ru967l2aj4n0rcs
```

Both runs take *Watch-only*. 'Full (seed + keys)' would add a bearer plate of master B's seed, which an automated run must not cut.

Wallet Policy

Bundle engraved. Also hand-engrave your ms1 share(s) - they are never sent over NFC.

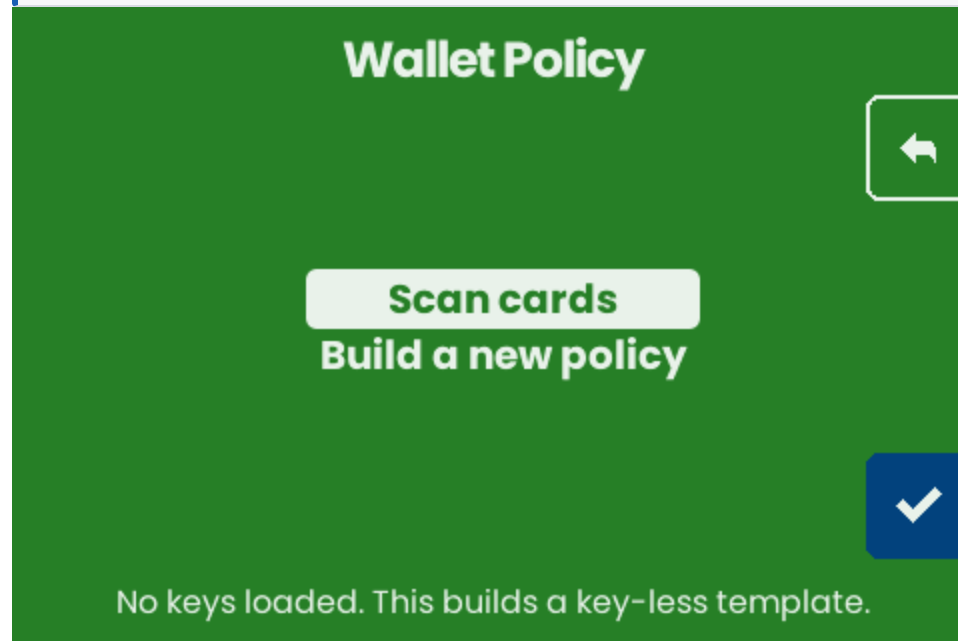


After the last plate: the bundle-engraved note, then the door again. (The ms1 reminder shows even when no share was cut -- F-463.)

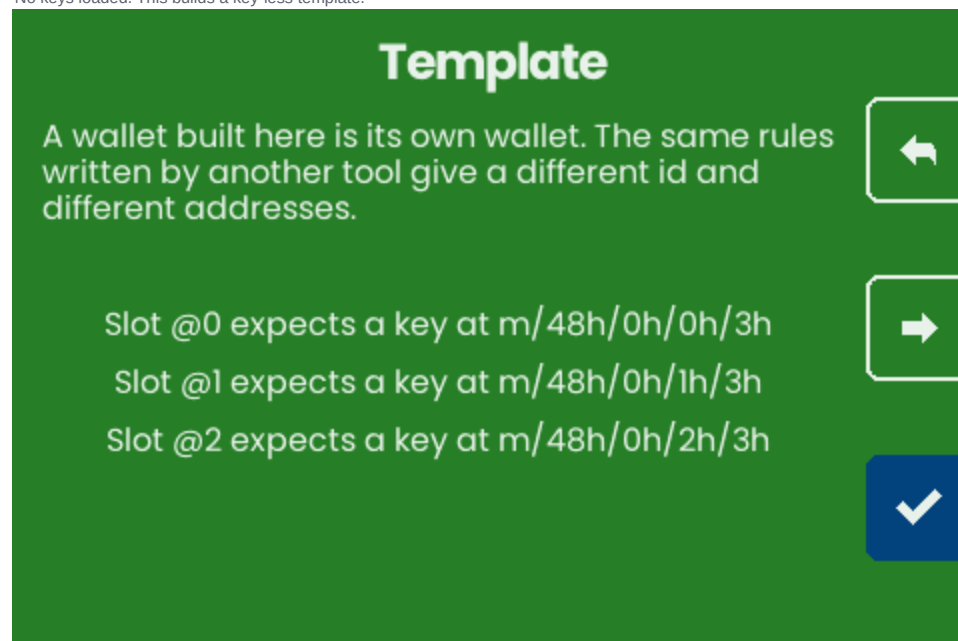
7 · The key-less arm — and the plate the operator cuts

With no payload loaded the door says so, and the composer builds a *key-less template*: Taproot, one 2-of-3 path, every slot given a distinct-account origin for a key to be minted later.

```
tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d547bfee9ace803ac0,sortedmulti_a(2,@0/48'/0'/0'/3'/<0;1>/*,@1/48'/0'/1'/3'/<0;1>/*,@2/48'/0'/2'/3'/<0;1>/*))
```



'No keys loaded. This builds a key-less template.'



Each unseated slot 'expects a key at' its own account: 0', 1', 2' under 48'/0'/.../3'.

Review

mk1 stub (template): e0863d3c

A wallet built here is its own wallet. The same rules written by another tool give a different id and different addresses.

Keyless template - no addresses.

Verify off-device.

Consent for a template: no addresses, verify off-device.

Plates To Cut

This engraves 1 plate.

mdl template: 1 plate (key-less wallet policy)

Each plate takes minutes to cut. Have that many blanks ready before you start: a set is only a backup when all of it exists.

mdl and mk1 plates carry error correction. A plain descriptor plate carries only its checksum, which finds a mistake but cannot fix one.

One plate.

Template-IDe0863d3ccac31a64d3b5e14b85ccd6c0

the platemd1fkzyyqq9qjtvyykjmprrj6tvyy49cqps8ys3psqcsmszu90h5wv13
characters56

Why the string is checked byte for byte and not merely verified. The device is chunk-form-always; a template this short encodes UNCHUNKED on the host by default, and `md verify`, `md decode` and `md inspect` accept both forms identically. The first draft of this journey's plan pinned the 47-character form; only the byte comparison could have told it from the 56-character plate. The transcript mints the oracle with `--force-chunked` and demonstrates the substitution:

```
tr; ONE path, 2-of-3; no lock, no hash; NO keys at all.
```

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md compose --wrapper tr --path 2of3
tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d547bfee9ace803ac0,sortedmulti_a(2,@0/48'/0'/0'/3'/<0;1>/*,@1/48'/0'/1'/3'/<0;1>/*,@2/48'/0'/2'/3'/<0;1>/*))
note: stdout is a keyless descriptor template (no keys)
[exit 0]
```

```
$ cat /scratch/code/shibboleth/wt-engrave-s4-emu/design/journeys/out/composer/keyless-tr.template
tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d547bfee9ace803ac0,sortedmulti_a(2,@0/48'/0'/0'/3'/<0;1>/*,@1/48'/0'/1'/3'/<0;1>/*,@2/48'/0'/2'/3'/<0;1>/*))
[exit 0]
```

--force-chunked, and this is the one place it is load-bearing rather than habitual. Without it this template encodes UNCHUNKED (47 characters) while the DEVICE is chunk-form-always. Both forms verify, inspect and decode identically -- same template, same ids -- so the substitution is invisible to every md verb, and only the byte comparison below catches it.

```
$ /scratch/code/shibboleth/descriptor-mnemonic/target/release/md encode tr(50929b74c1a04954b78b4b6035e97a5e078a5a0f28ec96d547bfee9ace803ac0,sortedmulti_a(2,@0/48'/0'/0'/3'/'<0;1>/*,@1/48'/0'/1'/3'/'<0;1>/*,@2/48'/0'/2'/3'/'<0;1>/*)) --force-chunked --group-size 0md1fkzyyqq9qjtvyykjmprrj6tvyy49cqps8ys3psqscsmzu90h5wvl3chunk-set-id: 0xb0884md1fkzyyqq9qjtvyykjmprrj6tvyy49cqps8ys3psqscsmzu90h5wvl3group size: 0separator: spacenote: stdout is a keyless descriptor template (no keys)[exit 0]
```

```
GATE PASS keyless md1 chunk count= 1GATE PASS keyless md1 string= md1fkzyyqq9qjtvyykjmprrj6tvyy49cqps8ys3psqscsmzu90h5wvl3... clipped; full text in transcript_composer.txt
```


8 · What this run actually established

The capture is a **comparison, not a photo shoot**. `shots_composer.js` is handed the host's digest, ids, stubs, addresses and engraved strings and throws if the device disagrees; the capture exits non-zero unless every leg matched and every shot arrived.

keyed-A21 shots · census 2 plate(s) · 7 engraved entries · 62 s

keyed-B21 shots · census 4 plate(s) · 9 engraved entries · 73 s

keyless8 shots · census 1 plate(s) · 1 engraved entry · 28 s

Rows are selected by *tapping* them where the layout draws them (`shTargets()` reads the frame's hit regions; the walk injects no button event the machine lacks). That is not a nicety: the first attempt at this walk found that the composer's pick lists had no touch targets at all, so on the device only a page's first row could be taken (the S4 walk's W-2, fixed before this run).

The controls

- `capture_composer.py --arm keyed --prove-it-can-fail` corrupts one character of one expected address and exits 0 only if the walk's failure *names that address*; a walk that fails earlier is reported INCONCLUSIVE.
- Substituting the unchunked key-less string for the expected one makes the key-less arm fail naming 56 versus 47 characters.

Reproducing

```
bash transcript_composer.sh > transcript_composer.txt 2>&1 # the host half, 27 gates
python3 capture_composer.py --arm both # rebuilds emu.wasm, drives three legs, writes shots/
python3 capture_composer.py --arm keyed --prove-it-can-fail
python3 build_pdf_composer.py # this document
```

What this journey does NOT show

- **No real plate.** The emulator's toolpath is read back; the physical key-less plate is the S4 device walk's, cut and read aloud at the machine.
- **No Full-mode secret plate,** no NFC seating, no on-device preimage derivation (spec §14).